When Does Force Feedback Help?

Predictive objectives and contact-onset force use in a controlled simulation study of robot insertion

Maria (Anh Duong Vo) · anhduongvo.m@gmail.com · revised version (v0.2), October 2026 · code, data and results: the `contact-state-reuse` repository, release v0.2.0

Abstract. Learned manipulation policies often receive force as one more input. This simulation study asks when force helps: as an input, as a target that a predictive auxiliary loss learns to predict, or both, and at which moments of an insertion. In a planar peg-in-slot task with a source domain and four physical and sensing shifts, every condition uses the same recurrent network (61k parameters) and the same successful demonstrations; conditions differ only in which signals the network receives and which losses are switched on (33 conditions, 5 training seeds each, 1,250 evaluation episodes per condition; intervals resample training runs and evaluation episodes). Predicting the next observation and the next 0.25 s of force raised success from 0.78 to 0.96 and lowered peak contact force from 11.4 to 5.5 N relative to behaviour cloning with the same inputs; relative to behaviour cloning with a tuned learning rate the difference was +0.07 success and -3.6 N. Force was most useful as a training target: predicting force without receiving it raised success by 0.22 [0.05, 0.41], whereas adding the live force reading to a model that already predicts force left success unchanged and lowered peak force by 1.4 to 2.4 N in the four domains without sensor faults (it raised it by 3.8 N when the sensor was faulty). Much of the force reduction came through the commanded compliance: when the evaluation harness fixed compliance, predictive and behaviour-cloning policies reached similar forces. Removing the force reading for 0.3 s at contact onset raised peak force by 1.8 N (source) and 2.2 N (tight fit), whereas equally long windows 0.3 s later, which removed a similar amount of force signal, had about a quarter of that effect and later windows none. With a second, non-privileged demonstrator and on a second task (hole search under large pose error), predictive objectives with a force target again gave the most reliable learned policies; force as an input was essential only on hole search, which requires reacting to contact. An exploratory representation analysis under a shift that requires new behaviour found no positive link between representation stability and adaptation, and predictive models protected against the loss of performance that small-data fine-tuning caused in behaviour cloning. The earlier version's finding that force as an input harms behaviour cloning did not replicate once data and architecture were matched. All results are from simulation.

Main results

1. **Predictive objectives improved reliability and lowered contact force, on identical data.** With the same network and the same successful demonstrations, predicting observations and force raised success by 0.18 [0.10, 0.27] and lowered peak force by 6.0 N [-8.7, -3.7] relative to behaviour cloning with force, in all five seeds (Section 5.1). The gain over a tuned behaviour-cloning baseline was smaller (+0.07 success, -3.6 N) but remained (Section 5.2).

2. **Force helped most as a training target.** Predicting force improved success even when force was not an input (+0.22). Given a force-predicting model, the live force input lowered contact force by 1.4 to 2.4 N without changing success, and it was a liability under sensor faults unless the model was trained with simulated faults (Sections 5.1, 5.5).
3. **The benefit was not explained by failure data, generic reconstruction, or the policy reading the predictive latent.** Adding failed episodes did not change the predictive models; prediction beat reconstruction of the current observation; prediction heads on a separate recurrent core worked as well as on the shared one (Section 5.2).
4. **The policy relied on force mainly at contact onset, and it lowered forces mainly by commanding softer compliance** (Sections 5.3, 5.4).
5. **The pattern held for a second demonstrator and a second task.** Force as a training target helped in all three settings; force as an input was essential only for the hole-search task, which requires reacting to contact (Section 5.6).
6. **Representation stability did not predict adaptation (exploratory).** Under a shift that lowers zero-shot success, learned latents were not more stable than an untrained network's, stability was not associated with adaptation gain, and the v0.1 fine-tuning dip turned out to be a small-data effect of behaviour cloning that the predictive models avoided (Section 5.7).

1. Motivation

In insertion, pressing and assembly, success is decided after the moment of contact. That is where cameras see least: the tool hides the part, millimetre misalignments are invisible, and friction or stiffness cannot be seen at all. Force and proprioception carry this information. Whether a learned policy can use it is less clear. Force-aware imitation methods report that concatenating force to a vision policy does not reliably help, and several add explicit machinery (contact predictors, phase schedulers, separate force controllers) to decide when force should matter [He et al., 2025; Wang et al., 2026; Fang et al., 2026].

This study asks a narrower question with a controlled design: **in which role, and at which moments, does force help a learned insertion policy?** A force signal can be an input that the policy reads, or a target that an auxiliary loss asks the policy's internal state to predict. These roles are usually changed together with the architecture, the data and the training budget, which makes the comparison hard to read. Here they are separated on one fixed network trained on identical data.

The analysis tools come partly from systems neuroscience, where decoding, cross-condition generalisation and perturbation are used to separate "the information is present" from "the system uses it" [Bernardi et al., 2020; Kornblith et al., 2019]. In this revision those tools are used only in an exploratory part (Section 5.7); the main contribution is the controlled behavioural comparison.

What changed in this revision. The first version (v0.1, Appendix C) compared models of matched parameter count but different trunk widths, trained behaviour cloning on successful episodes while prediction losses also saw failed episodes, used 3 seeds with intervals over evaluation episodes only, and framed the study around "contact-state reuse". Reviewers pointed out that these choices did not isolate the predictive objective, that the

uncertainty ignored variation between training runs, and that the reuse analysis could not test its hypothesis. All experiments in Sections 5.1 to 5.6 are new, use one fixed network, identical data and 5 seeds, and report intervals over training runs and episodes. The reuse and adaptation analysis was redesigned and is reported as exploratory.

2. Background

Force-aware imitation learning. FoAR gates a force branch with a future-contact predictor [He et al., 2025]. PhaForce schedules slow visual planning and fast force-based correction with a contact-aware phase predictor [Wang et al., 2026]. Force Policy separates a vision policy for free motion from a force policy for hybrid force-position control during contact [Fang et al., 2026]. ForceVLA adds force-aware experts to a vision-language-action model [Yu et al., 2025]. TacForeSight predicts future tactile latents conditioned on wrist force and torque [Zang et al., 2026]. A survey concludes that force is essential mainly for a small set of tasks, peg-in-hole insertion among them [Xie and Correll, 2025].

Predictive and auxiliary objectives. Lee et al. [2019] learned representations for contact-rich insertion by predicting contact-related signals from vision and force. Auxiliary prediction tasks are a standard way to shape the representation of a policy [Jaderberg et al., 2017]; latent dynamics models such as PlaNet [Hafner et al., 2019] learn predictive state representations for planning. Behaviour cloning from observation histories is known to latch onto shortcuts such as the previous action [de Haan et al., 2019; Wen et al., 2020].

Gap addressed. Most force-aware methods are compared with a few baselines, so the use of force, the architecture, the data and the training budget change together. This study fixes the network, the data and the budget, varies the role of force and the training losses in a factorial design, and tests where in time the trained policy depends on force.

3. Questions and hypotheses

The five hypotheses of v0.1 were written down before the first runs. In this revision their wording is kept, their test conditions are clarified, and an outcome is called "not supported" (the predicted effect was absent or opposite) or "inconclusive" (the interval was too wide to decide), never "falsified" on the basis of an interval that includes 0. H1 originally named force and proprioception; proprioception is never removed in the experiments, so H1 is tested for force only. Questions Q1 to Q4 were added in response to the review.

	Statement	Test
H1	Force improves success and safety, mostly under physical shifts	force input vs none, per model and domain (E)
H2	Predictive training produces latents that encode contact phase better than behaviour cloning	linear decoding of the contact mode (R)
H3	Contact-state representations are reused across a physical shift	phase-matched similarity against references and nulls (R)
H4	More reuse goes with faster, safer adaptation	gain over zero-shot vs reuse across trained models (R)
H5	Force matters specifically around first contact	force removed at onset vs later windows of equal length (T)
Q1	Does the predictive objective help on identical data, and is force needed as an input, as a target, or both?	factorial E
Q2	Is the gain due to failure data, generic auxiliary losses, or a weak baseline?	controls in E
Q3	How do predictive policies achieve lower forces?	behaviour metrics, fixed compliance (T)
Q4	Does the result depend on the demonstrator or the task?	G1, G2

4. Methods

4.1 Task and signals

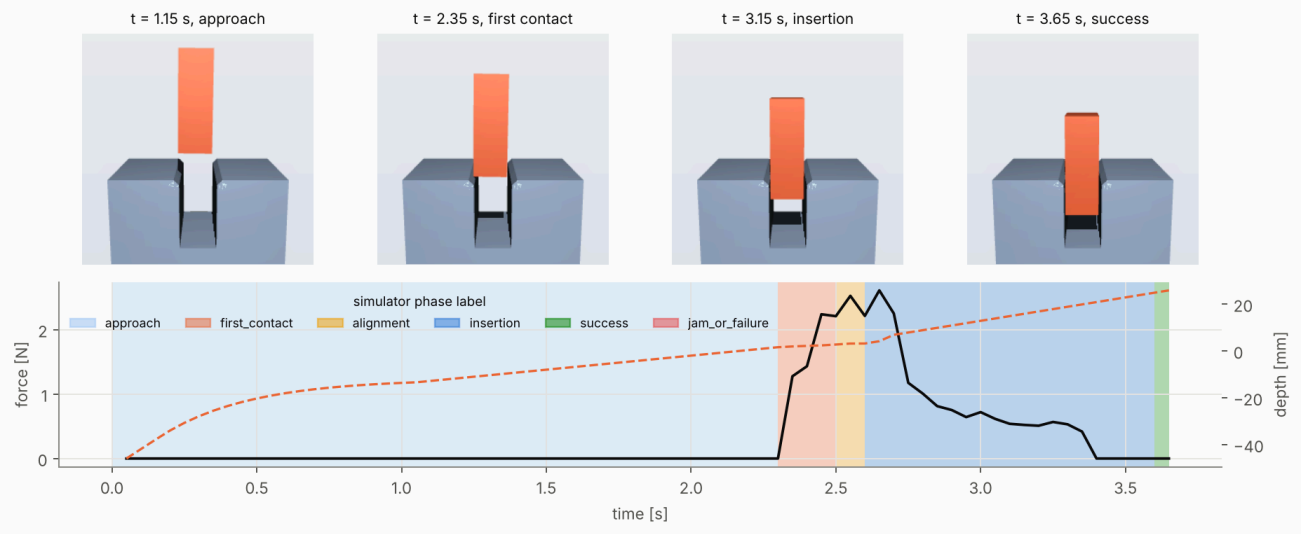


Figure 1. The planar insertion task. Top: rendered frames of one scripted demonstration. Bottom: true contact force (black) and insertion depth (dashed) over time, with the simulator-defined phase label shaded.

`ContactInsertionEnv` is a Gymnasium environment built on MuJoCo [Todorov et al., 2012; Towers et al., 2024]. A 20 mm x 60 mm peg moves in the x-z plane with three joints (tip x, tip z, rotation about y). Each joint is a PD actuator around a set-point; the action moves the set-point by at most 4 mm (0.03 rad) per 50 ms control step and sets a compliance level that scales the stiffness between 25 % and 100 % of its maximum. The slot is 40 mm deep with a 3 mm, 45-degree chamfer; success means 25 mm insertion depth with the tip inside the bore. Episodes end on success, on a jam (in contact for 1.5 s with mean force above 2 N and less than 0.5 mm of progress), on damage (contact force above 40 N) or after 8 s (160 control steps). Physics runs at 500 Hz with MuJoCo's implicit-fast integrator and elliptic

friction cones; contacts are soft (solref time constant 6 ms scaled by the contact-stiffness parameter).

The policy observes three groups of signals:

Group	Dim.	Content
exteroception	4	estimated slot entrance relative to the peg tip (x, z) and relative angle, with a fixed per-episode calibration offset (up to 2.5 mm), per-frame noise, and occlusion near the slot (frame dropped, last value held, validity flag 0)
proprioception	7	joint positions, joint velocities, current compliance level (joint efforts are excluded so that removing the force channel removes all contact-force information)
force	3	planar contact wrench on the peg, after a sensor model with noise, bias, delay and dropout

Force signal and force metrics. For every MuJoCo contact involving the peg, `mj_contactForce` gives the contact force in the contact frame; it is rotated to the world frame and summed over contacts. The force channel is (Fx, Fz) of this sum and the torque of the contact forces about the peg tip. It is therefore an idealised external contact wrench, not a model of a wrist force/torque sensor or of motor-current sensing. *Peak contact force*, the main safety metric, is the maximum over all 2 ms physics steps of an episode of the magnitude of the summed planar force. Because opposing contacts (for example both slot walls) partly cancel in a sum, I also report the *summed-contact peak*: the maximum over physics steps of the sum of the magnitudes of the individual contact forces. Force violation is a peak above 10 N.

Analysis labels. Each step receives one of six simulator-defined labels: approach, first contact (4 steps from first touch), alignment, insertion, success, jam or failure. They are measurement conventions used for analysis and to schedule interventions. In v0.2 no policy uses them, as input or as target (the v0.1 "simulator-supervised" condition is reported in Appendix C only).

Evaluated domains. Per-episode randomisation of slot position and tilt, clearance (0.6 to 1.2 mm), friction (0.2 to 0.5), peg mass, contact stiffness and sensor parameters defines the *source* domain. Four shifts move one factor outside that range: *shifted friction* (0.75 to 1.0), *tight clearance* (0.25 to 0.45 mm), *sensor fault* (force noise 1 to 2 N, bias 1 to 2 N, 2-step delay, 10 % dropout) and *occlusion* (70 to 90 % of frames dropped near the slot, higher pose noise). Together with the source domain these are the five evaluated domains. Two further settings were added in the revision: *search* (the pose estimate is offset laterally by 5 to 9 mm, so the peg lands on the top face and must slide to find the opening; used as a second task and as the adaptation shift) and composite domains such as *search + tight clearance*.

4.2 Demonstrators and data

Privileged demonstrator (main data). A scripted state machine with access to the true slot pose and the noise-free contact wrench approaches a point above the slot with a sampled aim error, descends until contact, and then complies laterally with the contact force while regulating the axial force to 5 N with a soft compliance setting. 20 % of episodes use perturbed strategies (large aim error, stiff pushing, uncorrected angle error) so that the data contain jams and high-force contacts. Executed actions carry DART noise (sigma 0.1 [Laskey et al., 2017]); the stored training target is the clean demonstrator action. 300 source-domain episodes, 84 % successful.

Non-privileged demonstrator (second demonstrator). The same state machine, but it perceives only the observation vector: the slot pose is estimated online from the noisy, biased, occludable pose estimate (exponential average in the world frame, held when a frame is invalid), and contact is detected from the noisy force reading. It is used as a simple non-privileged force-feedback controller (a baseline policy) and to generate a second training set (300 episodes, 90 % successful).

Search demonstrator (second task). The non-privileged demonstrator first tries to insert for 0.5 s after touch-down with a light (1 N) axial force; if the tip has not dropped by 1.5 mm, it slides along the top face towards +x, reversing after 12 mm, until the tip drops, then inserts. The search direction is fixed, so the behaviour is a function of the observation history and can be imitated. 300 episodes in the *search* domain, 96 % successful.

Training data used by every loss. In v0.2 all losses (behaviour cloning and prediction) are trained on the successful episodes only (252 of 300; 15 % of all episodes are held out for validation before filtering, leaving 216 successful episodes for training and 36 for validation), so every objective sees exactly the same data. Including the failed episodes in the prediction losses (the v0.1 setting) is a separate experimental factor (Section 5.2).

Datasets generated with the same seed in different domains are paired episode by episode (same geometry, start pose, noise streams and demonstrator randomness). The random streams of the original domains are unchanged by the v0.2 additions; regenerated v0.1 datasets are bit-identical.

4.3 One network, different training signals

Every v0.2 model is the same network: a fusion encoder (one linear embedding and LayerNorm per modality; excluded modalities are multiplied by a zero mask, so all input ablations share parameters), a GRU with 66 hidden units over the encoded observation and the previous action, an action head, and three prediction heads that read the same latent z_t: next observation (exteroception and proprioception, 11 values), future force over the next 5 steps (Gaussian mean and log-variance) and contact mode (unused in v0.2). Total 61,469 parameters for every condition. Conditions differ only in which loss terms are switched on, whether force is an input, and the training data:

Factor	Levels
force as input	yes / no (zero mask)
next-observation prediction	weight 0 / 1
future-force prediction (force as a training target)	weight 0 / 1

With both prediction weights at 0 the model is a sequence behaviour-cloning policy (the prediction heads exist but receive no gradient). Controls reuse the same network with: a reconstruction head for the *current* observation instead of prediction (adds a head), prediction heads on a detached latent (stop-gradient), prediction heads on a second GRU that shares only the encoder (separate core; adds a GRU), other loss weights (0.1, 0.3, 3), no previous-action input, transformed force inputs (signed log, or a binary contact flag at 1 N), fault-aware augmentation, other learning rates and updates, demonstrations without DART noise, and a behaviour-cloning network with a wider GRU (94 units, 116k parameters).

What "world model" means here. The predictive models learn to predict future observations and forces from the history of observations and executed actions, and the policy reads the same latent. They are not used to evaluate alternative future actions, to plan, or to train the policy on imagined trajectories. "Predictive auxiliary learning for a recurrent policy" is the precise description; I use "predictive model" for short.

Losses and their scale. Actions are in [-1, 1]; observation and force targets are standardised per feature with the training-set statistics. The action and observation terms are mean squared errors and the force term is a Gaussian negative log-likelihood; each is a mean over feature dimensions and over valid time steps. Future-force targets are the *sensor readings* of the next 5 steps (with the source-domain noise and bias, not simulator ground truth); steps beyond the end of an episode are masked. Equal weights do not imply equal gradient contributions, so the trainer logs, every 100 updates, the gradient norm of each weighted term on the shared encoder and GRU (Section 5.2).

Training. AdamW, learning rate 1e-3 with cosine decay to 5e-5, weight decay 1e-4, 16 full episodes per batch, 1,500 updates, gradient clipping 1.0; 15 % of episodes held out for validation; the final checkpoint is used (no checkpoint selection). About 80 s per model on one CPU core.

4.4 Experiments

	Question	Design
E	Which training signal makes force useful?	2 × 2 × 2 factorial (force input, observation target, force target) on the fixed network and identical data, plus failure-data, auxiliary-loss, fault-aware and behaviour-cloning-diagnostic conditions; 33 conditions x 5 seeds, 50 evaluation episodes per domain
T	When is force used?	evaluation-time interventions on E checkpoints: force reading removed for 0.3 s starting at first contact, 0.3, 0.6 or 1.2 s later, or at a random time; held at the last pre-contact value instead of zeroed; kept only in the onset window; removed for the whole episode; scaled by 0.5 or 2. Also compliance fixed by the harness. 60 episodes per domain, source and tight clearance
L	Is contact information localised in the latent?	25 % of latent dimensions clamped to their mean (contact-ranked, movement-ranked, random) during control; 5 seeds, 40 episodes per cell
G1	Does it depend on the demonstrator?	key conditions trained on the non-privileged demonstrator
G2	Does it hold on a second task?	key conditions trained from scratch on hole search
R	Representation stability and adaptation (exploratory)	E checkpoints adapted to the search shift (severities 0.5, 0.75, 1) with 5 to 40 demonstrations; source fine-tuning control; tight-clearance fine-tuning at two learning rates; reuse scores against reference representations

The first-contact time used to schedule interventions in T comes from the simulator; the policy never receives it. For every intervention the harness records whether the window was reached before the episode ended and how much force signal was removed (sum of the magnitudes of the readings that were replaced).

4.5 Evaluation and statistics

Every model is evaluated on the same evaluation episodes (seed 123; never used for training data), so comparisons are paired by episode. In E: 50 episodes x 5 domains x 5 seeds = 1,250 episodes per condition, 41,250 in total (plus 3 extra sensor corruptions for 3

conditions); T: 60 episodes x 2 domains x 5 seeds per intervention; G1 and G2: 50 episodes per domain and seed.

Uncertainty at the level of training runs. Results form a crossed design: every condition is trained with the same 5 seeds and evaluated on the same episodes. Means are reported with 95 % intervals from a crossed ("pigeonhole") bootstrap [Owen, 2007] that resamples training seeds and evaluation episodes independently (2,000 resamples). Paired differences are computed cell by cell (same seed, same episode) and resampled the same way, so the intervals include variation between training runs and between episodes and keep the pairing. I also give per-seed means and the number of seeds favouring each condition. With 5 seeds these intervals are wide when runs disagree, which is the intended behaviour [Agarwal et al., 2021].

First-contact anticipation. For force-predicting models: AUROC of the predicted future force magnitude for "first contact within the next 3 steps", computed only on steps *before* first contact (frames already in contact are excluded), compared with two simple scores on the same steps: the current force reading and the negative estimated distance to the slot entrance.

4.6 Representation analysis (exploratory)

- **Probes.** Logistic regression decoding the contact mode from frozen latents. Within-source: 5-fold cross-validation grouped by episode. Held-out pairs: the episode pairs are split into 5 folds; the probe is fitted on the source members of the training pairs and tested on both the source and the shifted members of the held-out pairs, with balanced accuracy over the same classes for both. The v0.1 cross-domain probe (fit on all source episodes, test on all shifted episodes of the same pairs) is reported for comparison.
- **Phase-matched CKA and reuse score.** For each phase, each paired episode's latents in that phase are resampled to 4 points in both domains and compared with linear CKA [Kornblith et al., 2019]. The null permutes the episode pairing within a phase (50 permutations). Reuse in phase p is (CKA_p - mean null_p) / (1 - mean null_p), averaged over approach, first contact, alignment and insertion; I report the null distribution (5th to 95th percentile), the raw difference CKA - null, and sensitivity to resampling (2 or 8 points) and to excluding the approach phase. Intervals: leave-one-episode-out jackknife (an episode bootstrap is biased upwards for CKA because duplicated episodes add identical rows to both sets).
- **Reference representations.** The same score for the standardised raw observation, proprioception alone, a task-progress clock (time step and time since first contact) and the untrained network with the same architecture and seed.
- **Adaptation.** Fine-tuning always starts from the pre-trained checkpoint; budgets are nested subsets of one fine-tuning set (the 5 episodes are part of the 10, and so on); 300 updates, learning rate 3e-4 (also 1e-4 on tight clearance); final checkpoint. Adaptation is measured as the gain over zero-shot success: final gain, area under the gain curve, and the number of demonstrations needed to reach 0.9 success. Correlations with reuse are computed per trained model (n = 20, reuse and gain averaged over severities) and per severity.

5. Results

5.1 Which training signal makes force useful (Q1, H1)

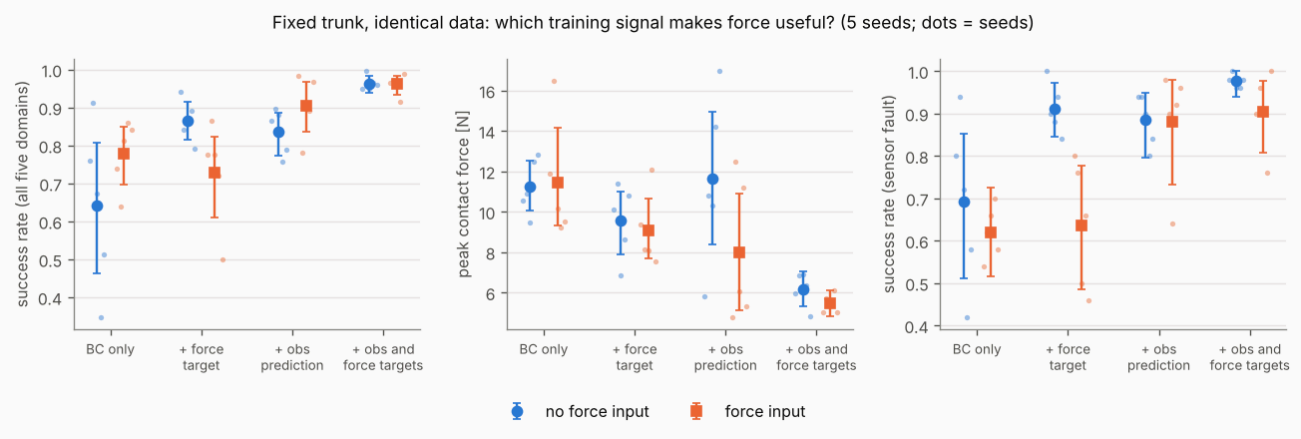


Figure 2. Experiment E, the 2 x 2 x 2 factorial on one fixed network trained on the same successful demonstrations. Large markers: mean over 5 seeds x 250 episodes (50 per domain) with crossed seed x episode 95 % bootstrap intervals; small dots: individual seeds. Left: success pooled over the five domains. Middle: peak contact force. Right: success in the sensor-fault domain.

Condition	Force input	Obs. target	Force target	Success	Per-seed success	Peak force [N]	P(peak > 10 N)
BC				0.64 [0.46, 0.81]	0.76, 0.91, 0.67, 0.35, 0.51	11.2 [10.0, 12.5]	0.50
BC + force	x			0.78 [0.70, 0.85]	0.74, 0.64, 0.81, 0.86, 0.84	11.4 [9.3, 14.1]	0.40
force target			x	0.87 [0.82, 0.91]	0.94, 0.84, 0.86, 0.89, 0.79	9.5 [7.9, 11.0]	0.37
force input + target	x		x	0.73 [0.61, 0.82]	0.78, 0.86, 0.78, 0.72, 0.50	9.0 [7.7, 10.7]	0.33
obs prediction		x		0.84 [0.77, 0.89]	0.86, 0.90, 0.88, 0.76, 0.79	11.6 [8.4, 15.0]	0.44
obs prediction + force	x	x		0.91 [0.84, 0.97]	0.98, 0.78, 0.90, 0.89, 0.97	8.0 [5.1, 10.9]	0.25
obs + force targets		x	x	0.96 [0.94, 0.98]	0.95, 1.00, 0.95, 0.96, 0.96	6.2 [5.3, 7.0]	0.17
obs + force targets + force	x	x	x	0.96 [0.93, 0.98]	0.96, 0.97, 0.97, 0.92, 0.99	5.5 [4.8, 6.1]	0.13

Table 1. Experiment E, pooled over the five domains (1,250 episodes per row). The full table with all 33 conditions and every domain is in `results/revision/E_conditions.csv`.

Observation prediction helped in every comparison. Adding the next-observation target raised success by 0.20 [0.07, 0.33] without force, 0.13 [0.06, 0.20] with force as input, 0.10 [0.04, 0.15] with a force target and 0.23 [0.14, 0.37] with both, in 4 or 5 of the 5 seeds each time.

Force helped most as a training target. Predicting the next 5 force readings, without receiving force as an input, raised success by 0.22 [0.05, 0.41] (4 of 5 seeds), by 0.22 to 0.27 in the source, friction, tight-fit and sensor-fault domains separately and by 0.15 [-0.08,

+0.40] under occlusion; with observation prediction it added 0.13 [0.07, 0.19] success and changed peak force by -5.4 N [-9.2, -2.0] (success higher in 5 of 5 seeds, peak force lower in 4 of 5). A model that predicts observations and force but never sees force reached 0.96 success and 0.98 under sensor faults.

Force as an input mattered less. For behaviour cloning, adding force changed success by +0.14 [-0.10, +0.38], an inconclusive result driven by large seed-to-seed variation of the model without force (0.35 to 0.91). This does not reproduce v0.1, where force as an input lowered behaviour-cloning success by 0.07 to 0.23; that comparison used different trunk widths and different effective training data (Appendix C). For the model that predicts observations and force, adding force as an input left success unchanged (0.00 [-0.04, +0.03]) and lowered peak force by 1.4 N (source), 2.4 N (friction), 1.9 N (tight fit) and 1.6 N (occlusion), all intervals excluding 0, while under sensor faults it changed success by -0.07 [-0.16, -0.00] and peak force by +3.8 N [2.0, 5.4]. One cell does not fit the pattern: with a force target but no observation target, adding force input lowered success (-0.14 [-0.24, -0.04]); I have no explanation for this interaction.

Anticipating contact. On steps *before* first contact, the force head of the full model predicted contact within the next 0.15 s with AUROC 0.96 (source; 0.92 to 0.98 across domains), against 0.52 for the current force reading and 0.87 for the estimated distance to the slot on the same steps (`E_auroc.csv`). The predicted force therefore carries anticipatory information beyond the pose estimate.

H1 (force helps, mostly under shifts) is not supported as stated. The effect of force depended on its role and on the training objective, not mainly on the shift; force inputs helped least, and were harmful, under the one shift that corrupts the force signal itself.

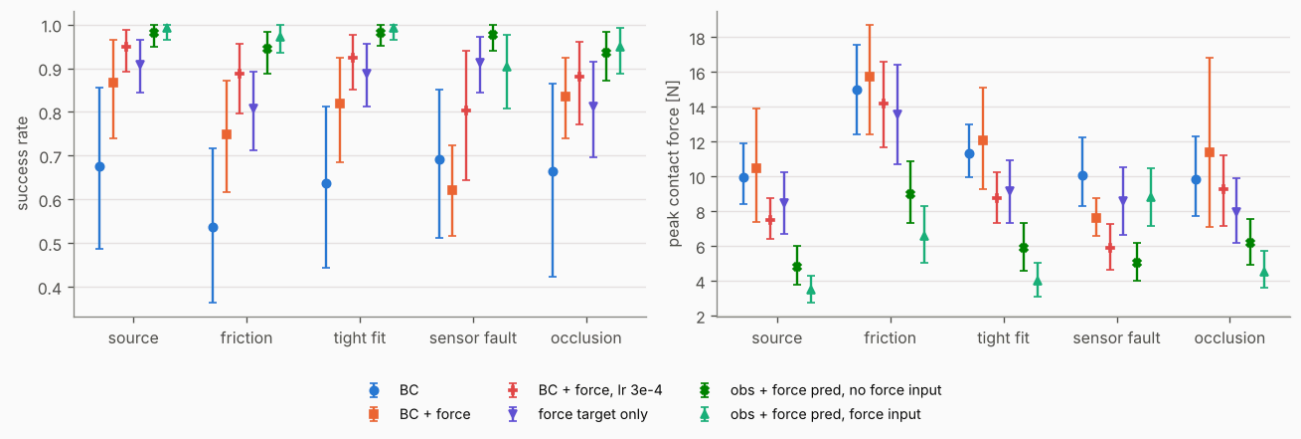


Figure 3. Success and peak contact force per domain for selected conditions of experiment E (mean and crossed 95 % interval, 5 seeds x 50 episodes).

5.2 What produces the gain (Q2)

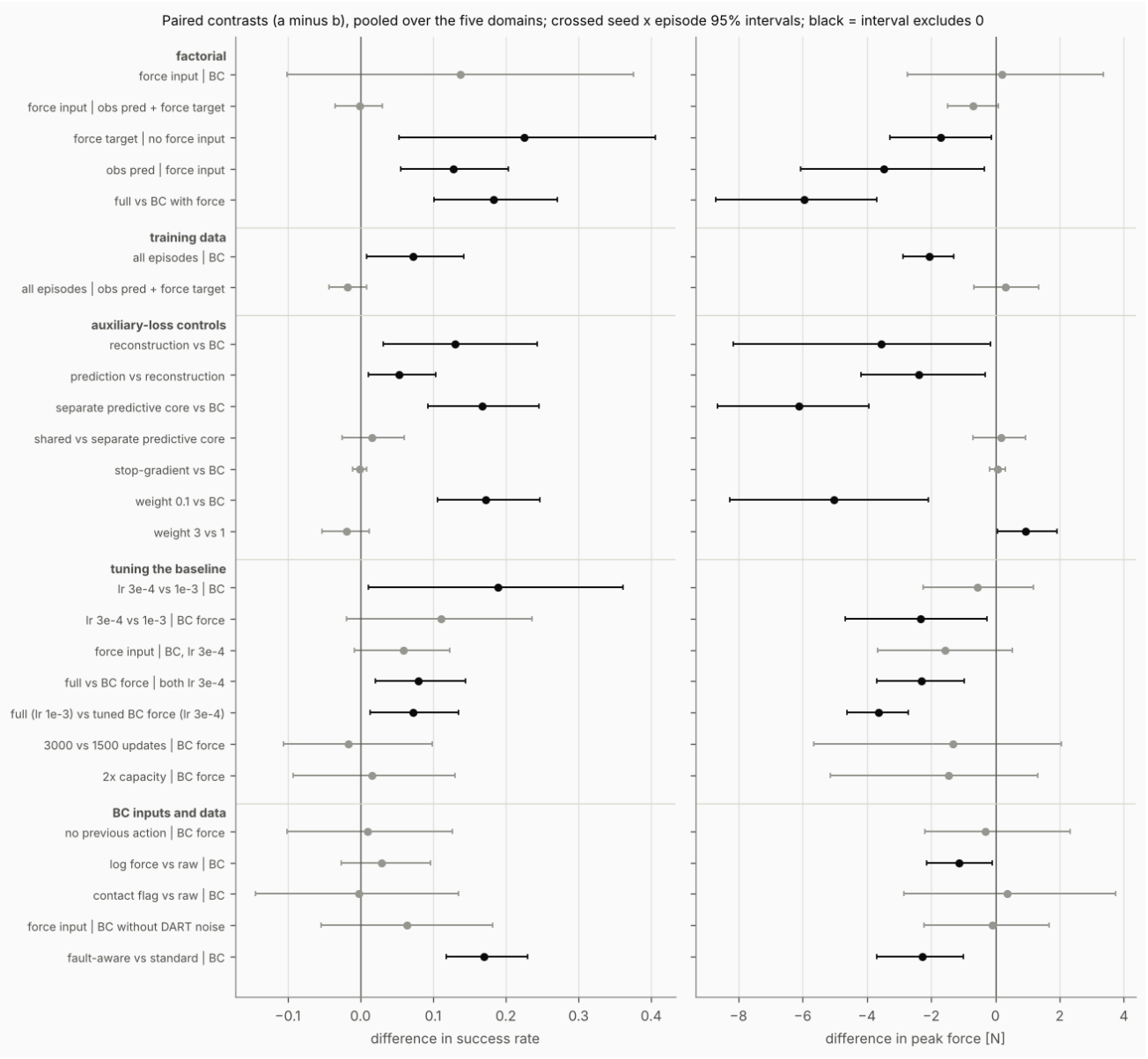


Figure 4. Paired contrasts in experiment E (condition a minus condition b), pooled over the five domains. Each difference is computed on matched seed and episode and resampled over both; black: the 95 % interval excludes 0. "Full" is the model with force input and observation and force targets.

Failure data. In v0.1 the prediction losses also saw failed episodes, which behaviour cloning did not. Training the prediction losses on all episodes changed nothing for the predictive models (-0.02 [-0.04, +0.01] success, +0.3 N [-0.7, +1.3]), so the predictive gain does not come from exposure to failures. For behaviour cloning, including failed episodes (which only changes batch composition and input normalisation, because the action loss still uses successful episodes only) raised success by 0.07 [0.01, 0.14] and lowered peak force by 2.0 N. A plausible reason is that the normaliser fitted on failure-containing data compresses large force readings (Section 5.3).

Generic auxiliary losses. Reconstructing the *current* observation as an auxiliary task helped behaviour cloning (+0.13 [0.03, 0.24], -3.6 N), but prediction helped more: prediction minus reconstruction was +0.05 [0.01, 0.10] success (5 of 5 seeds) and -2.4 N [-4.2, -0.3] peak force (4 of 5 seeds). Success was flat over a 30-fold range of auxiliary weights (0.1 to 3; weight 0.1 still gave +0.17 over behaviour cloning); peak force was about

1 N higher at weights 0.1 and 3 than at 1. The auxiliary terms dominate the gradient on the shared encoder and GRU: averaged over training, the gradient norm of the force term was 0.56 and of the observation term 0.24, against 0.017 for the action term (at weight 0.1: 0.05, 0.02 and 0.014).

Does the policy need to read the predictive latent? With the prediction heads on a separate GRU that shares only the encoder with the policy, the gain over behaviour cloning was the same as with a shared GRU (+0.17 [0.09, 0.25] success, -6.1 N [-8.6, -3.9]; shared minus separate +0.02 [-0.02, +0.06] and +0.2 N [-0.7, +0.9]). With the heads on a detached latent (stop-gradient), the model behaved like behaviour cloning (0.00 [-0.01, +0.01]), as it should, since then the auxiliary losses cannot shape any shared parameter. So in this setting the predictive losses act through the shared observation encoder; the recurrent latent the action head reads does not itself need to be predictive.

A tuned baseline. Behaviour cloning was sensitive to the learning rate: 3e-4 instead of 1e-3 raised success by 0.19 [0.01, 0.36] without force and by 0.11 [-0.02, +0.24] with force (lowering peak force by 2.3 N), while 1e-4, twice as many updates, a wider GRU (94 units, 116k parameters, about twice the parameters), removing the previous-action input, or removing DART noise had no measurable effect. With the same learning rate (3e-4) for both, the predictive model still differed from behaviour cloning with force by +0.08 [0.02, 0.14] success and -2.3 N [-3.7, -1.0] peak force, and the default predictive model differed from the tuned baseline by +0.07 [0.01, 0.13] and -3.6 N [-4.6, -2.7] (except in the sensor-fault domain, Section 5.5). Adding simulated sensor faults to the force input during training (Section 5.5) was the most effective regulariser for behaviour cloning (+0.17 [0.12, 0.23]).

Answer to Q2. The predictive gain survives identical data, a reconstruction control, a weight sweep, a baseline with about twice the parameters and a tuned baseline. Most of the *success* difference to behaviour cloning can be closed by other regularisers (lower learning rate, reconstruction, input-noise augmentation); none of them matched the predictive models' contact forces.

5.3 Why force as an input is fragile for behaviour cloning

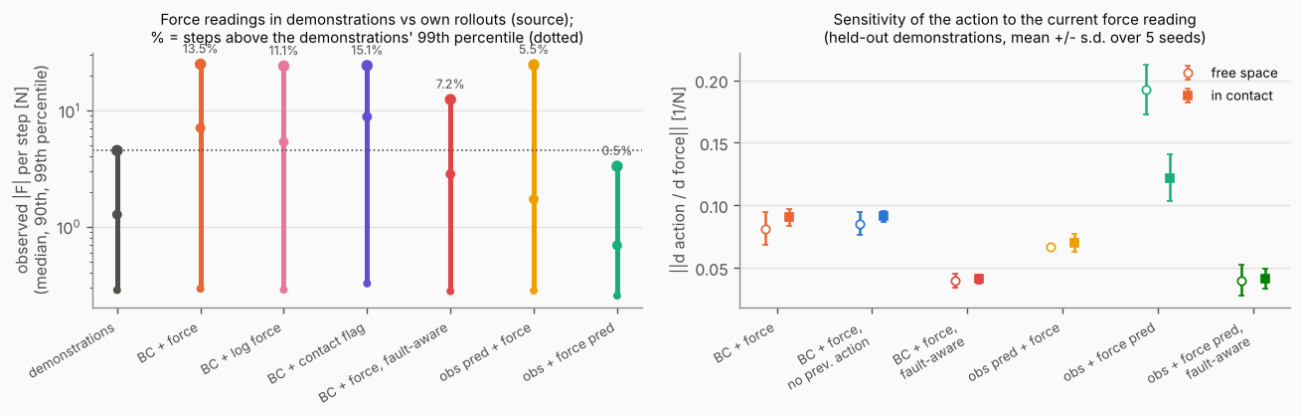


Figure 5. Left: distribution of observed force readings per step in the successful training demonstrations and in each policy's own rollouts (source domain, 5 seeds x 30 episodes); percentages are the share of steps above the 99th percentile of the demonstrations (4.5 N). Right: gradient norm of the action with respect to the current force reading on held-out demonstration steps.

In the successful demonstrations the force reading is small: median 0.28 N, 99th percentile 4.5 N, and above 1 N in 13 % of steps (89 % during alignment). Behaviour cloning with force spent 13.5 % of its own rollout steps above that 99th percentile (its own 99th

percentile was 25 N, maximum 30 N); the force-predicting model spent 0.5 %. So the behaviour-cloning policy drives itself into contact states that the demonstrations do not contain, where its force input is far outside the training range, a form of the covariate shift known for behaviour cloning [Laskey et al., 2017]. A signed-log transform of the force lowered peak force by 1.1 N [0.1, 2.1] but did not change success; a binary contact flag changed neither.

The suggestion in v0.1 that behaviour cloning "over-reacts" to force is not supported: on held-out demonstration steps, the action of the behaviour-cloning policy was *less* sensitive to the force reading (0.08 to 0.09 per N) than that of the force-predicting model (0.12 in contact, 0.19 in free space). Scaling the reading by 2 during evaluation *lowered* behaviour cloning's peak force by 1.9 to 2.4 N (experiment T), which also does not fit an over-reaction account.

5.4 When force is used and how forces are lowered (H5, Q3)



Figure 6. Experiment T, evaluation-time interventions on the E checkpoints (5 seeds x 60 episodes per cell). Top: paired change in peak force relative to the intact force reading. Bottom: force signal removed by the intervention (sum of the magnitudes of the replaced readings per episode, log scale). Windows are 6 steps (0.3 s); "+0.3 s" starts 6 steps after first contact.

Intervention (predictive model, source)	Change in peak force [N]	Force signal removed [N]	Window reached
removed at onset (0 to 0.3 s)	+1.8 [1.1, 2.6]	7.9	0.97
removed 0.3 to 0.6 s after onset	+0.4 [0.2, 0.8]	7.3	0.96
removed 0.6 to 0.9 s after onset	+0.1 [-0.0, +0.2]	5.1	0.91
removed 1.2 to 1.5 s after onset	0.0 [0.0, 0.1]	1.2	0.63
removed at a random time	0.0 [-0.3, +0.2]	2.5	0.84
held at the last pre-contact value at onset	+2.3 [1.5, 3.2]	8.1	0.97
kept only at onset, removed elsewhere	+0.7 [-0.1, +1.6]	43.8	1.00
removed for the whole episode	+4.0 [2.6, 5.5]	96.9	1.00

Table 2. Experiment T for the model with force input and observation and force targets, source domain (intact force: 3.3 N). Tight fit: onset +2.2 [1.2, 3.3], +0.3 s +0.6 [0.3, 0.9], whole episode +5.3 [3.6, 7.2].

Onset versus later contact. Removing the force reading for the first 0.3 s of contact raised peak force by 1.8 N; a window of the same length starting 0.3 s later removed a similar amount of force signal (7.3 vs 7.9 N summed) but raised peak force by only 0.4 N, and later windows almost none. The onset window minus the +0.3 s window was 1.4 N [0.7, 2.1] (source) and 1.6 N [0.6, 2.7] (tight fit). Keeping force *only* during the onset window and removing it everywhere else cost 0.7 N, against 4.0 N for removing it everywhere (difference 3.3 N [2.4, 4.2]). Replacing the onset readings by the last pre-contact value instead of zero had a similar or slightly larger effect (+2.3 vs +1.8 N in the source domain, equal in the tight fit), so the result reflects missing information, not a false "zero force" reading. Success changed little under any 0.3 s window (0.99 to 1.00). The tuned behaviour-cloning policy and the predictive model without a force target showed the same pattern with smaller and less clearly separated effects (onset +1.2 to +1.3 N; onset minus +0.3 s between +0.6 and +1.0 N).

These are contrasts between interventions on a closed-loop system; they do not add up to a decomposition of "the contribution of force", and v0.1's statement that the onset window "accounts for 85 to 94 % of the full effect" is withdrawn. **H5 is supported in the following sense:** the trained policies were particularly sensitive to losing the force reading during the first 0.3 s of contact, more than to losing a similar amount of force signal shortly afterwards.

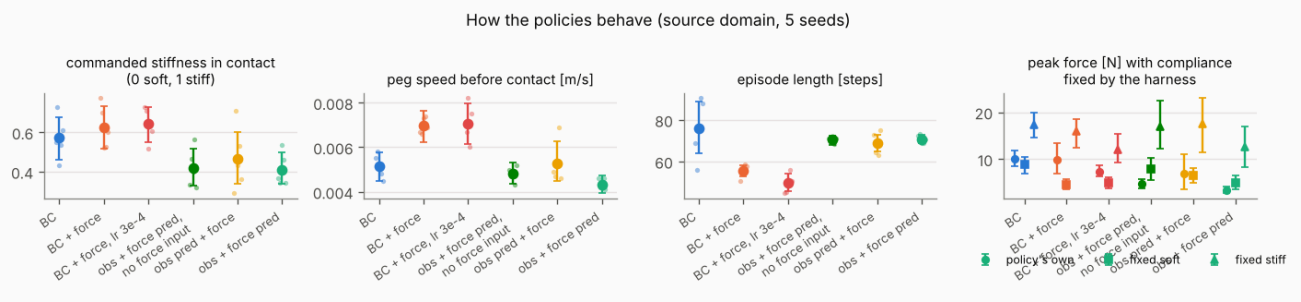


Figure 7. Behaviour in the source domain (5 seeds; dots are seeds). Commanded stiffness during contact (0 = softest compliance setting), peg speed in the last 5 steps before first contact, episode length, and peak force when the harness overrides the compliance command with a fixed soft (-0.5, the demonstrator's contact setting) or stiff (+0.5) value.

How forces are lowered (Q3). The predictive policies commanded softer compliance during contact (stiffness 0.41 vs 0.63 for behaviour cloning with force), approached the slot more slowly (4.3 vs 7.0 mm/s) and took longer (71 vs 55 steps, 3.5 vs 2.7 s to success). When the harness fixed the compliance command to the demonstrator's soft setting, behaviour cloning with force reached 4.5 N and the predictive model 4.9 N in the source domain (difference +0.4 N [-1.1, +2.3]); the predictive model kept a success advantage (+0.11 [0.01, 0.20]). So most of the force difference between these two policies is explained by the compliance each policy chooses. The predictive model's own compliance choice (3.3 N) was better than any fixed setting, i.e. it modulates compliance rather than simply staying soft. Without force input, behaviour cloning under fixed soft compliance still reached 8.9 N, so force input matters once compliance is controlled.

5.5 Sensor faults and fault-aware training

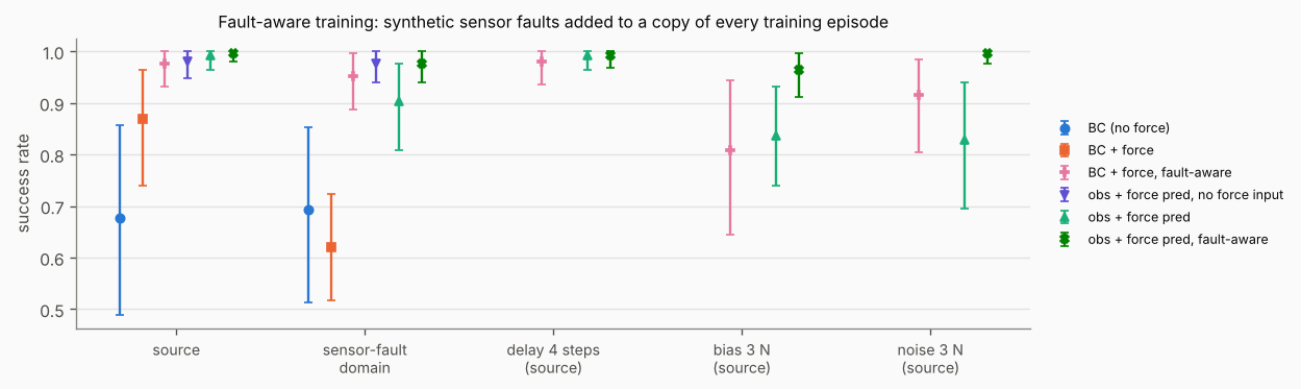


Figure 8. Success under sensor faults. Fault-aware models were trained with a copy of every training episode whose force input carries the sensor-fault model of the evaluation domain (noise 1 to 2 N, bias 1 to 2 N, 2-step delay, 10 % dropout); prediction targets keep the original readings. The three right columns are stronger faults not used in training, applied to the source domain.

Under the sensor-fault domain the predictive model with force input reached 0.90 success and 8.8 N, against 0.98 and 5.0 N for the same objective without force input. Fault-aware training removed this cost (0.98, 4.8 N; +0.07 [0.00, 0.17] success, -4.0 N [-6.0, -1.9]) without changing the other domains (+0.02 [-0.01, +0.04] pooled), and it also held under stronger faults not seen in training: bias of 3 N (0.96 vs 0.84 success, 5.2 vs 13.6 N) and noise of 3 N (1.00 vs 0.83). The fault model used for training matches the evaluated fault domain, so the first comparison is not a test of robustness to unknown faults; the 3 N bias and noise cases are a partial test. For behaviour cloning, the same augmentation raised success in every domain (+0.17 pooled), consistent with Section 5.3.

Deployment implication. Force input added safety margin only when the sensor was healthy or the model had been trained with simulated faults. The safest configurations in this study were a model that predicts force but does not read it (0.98 under faults) and one that reads it after fault-aware training (0.98).

5.6 A second demonstrator and a second task (Q4)

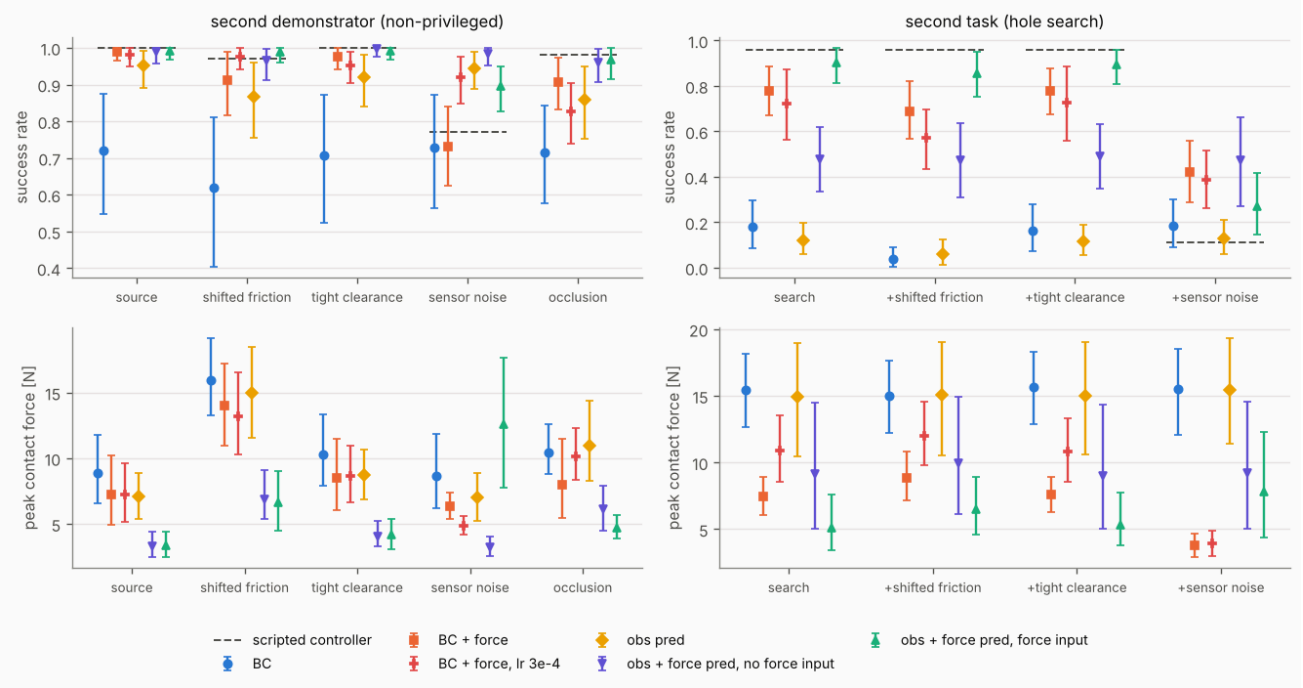


Figure 9. Left: the original task with demonstrations from the non-privileged demonstrator (G1). Right: the hole-search task trained from scratch (G2). Mean and crossed 95 % interval over 5 seeds x 50 episodes; dashed lines: the scripted non-privileged controller (100 episodes).

Pooled over domains	G1 success	G1 peak force [N]	G2 success	G2 peak force [N]
BC	0.70 [0.55, 0.84]	10.9	0.14 [0.08, 0.20]	15.4
BC + force	0.90 [0.86, 0.94]	8.8	0.67 [0.58, 0.75]	6.9
BC + force, lr 3e-4	0.93 [0.90, 0.96]	8.8	0.60 [0.48, 0.71]	9.4
obs + force targets, no force input	0.98 [0.96, 1.00]	4.7	0.48 [0.34, 0.62]	9.3
obs + force targets + force input	0.97 [0.95, 0.98]	6.3	0.73 [0.67, 0.79]	6.2
scripted controller (G1: non-privileged; G2: search)	0.94	3.3	0.75	4.4

Table 3. G1 and G2, pooled over five (G1) or four (G2) evaluated domains; scripted controllers: mean over the same domains (100 episodes each). G2 domains: search alone and combined with friction, tight fit and sensor faults.

Second demonstrator (G1). With demonstrations from the non-privileged demonstrator, whose own actions depend on the noisy force reading, force as an input helped behaviour cloning clearly (+0.20 [0.05, 0.36]). The model that predicts observations and force without receiving force was again the best learned policy (0.98 success, 4.7 N; +0.28 [0.16, 0.41] and -6.1 N [-7.7, -4.7] relative to behaviour cloning), and live force input added nothing in success (-0.01 [-0.03, +0.01]) and cost 9.4 N under sensor faults. Against behaviour cloning with force at the same learning rate, the full model had equal success (0.00 [-0.07, +0.06]) and lower peak force (-3.7 N [-5.7, -1.7]).

Second task (G2). Hole search needs contact feedback: without force input, the policies without a force target failed in most episodes (0.11 to 0.18), and force input raised behaviour cloning to 0.67. Even here, predicting force without receiving it raised success from 0.14 to 0.48 (+0.34 [0.16, 0.52]), and the full model was the best learned policy (0.73; 0.90 in the search domain without additional shifts), ahead of behaviour cloning with force (+0.07 [0.00, 0.13]) and of behaviour cloning with force at the same learning rate (+0.11 [0.01, 0.22], -4.7 N [-6.4, -2.9]). Under sensor faults all force-using policies failed often (0.14 to 0.42).

Scripted baselines. The non-privileged scripted controller, which uses the same observations as the learned policies, was a strong baseline on the original task (0.97 to 1.00 success and 2 to 4 N in the four fault-free domains; 0.77 under sensor faults), and its search variant was strong on hole search (0.96; 0.11 under sensor faults). The learned policies did not exceed it except under sensor faults, where the model that does not read force (0.98) was clearly more robust. This study compares learned policies with each other; it does not claim that learning beats a well-designed controller on a task that controller was designed for.

Answer to Q4. The main pattern held for a second demonstrator and a second task: predictive objectives with a force target gave the most reliable learned policies, and force as a training target helped in all three settings. The value of force as an *input* depended on the setting: small given a force target on the original task, essential for hole search.

5.7 Exploratory: representation stability and adaptation (H2, H3, H4)

This section is exploratory. It asks whether the learned representations are stable across a shift and whether stability goes with faster adaptation, using a shift chosen so that adaptation can be measured: the *search* shift (pose estimate offset by 5 to 9 mm at severity 1), which lowers zero-shot success of every E model while the search demonstrator still solves it (0.96). The pre-trained models are the E checkpoints of four families (behaviour cloning with force at learning rates 1e-3 and 3e-4, observation prediction with force, and the full model), 5 seeds each.

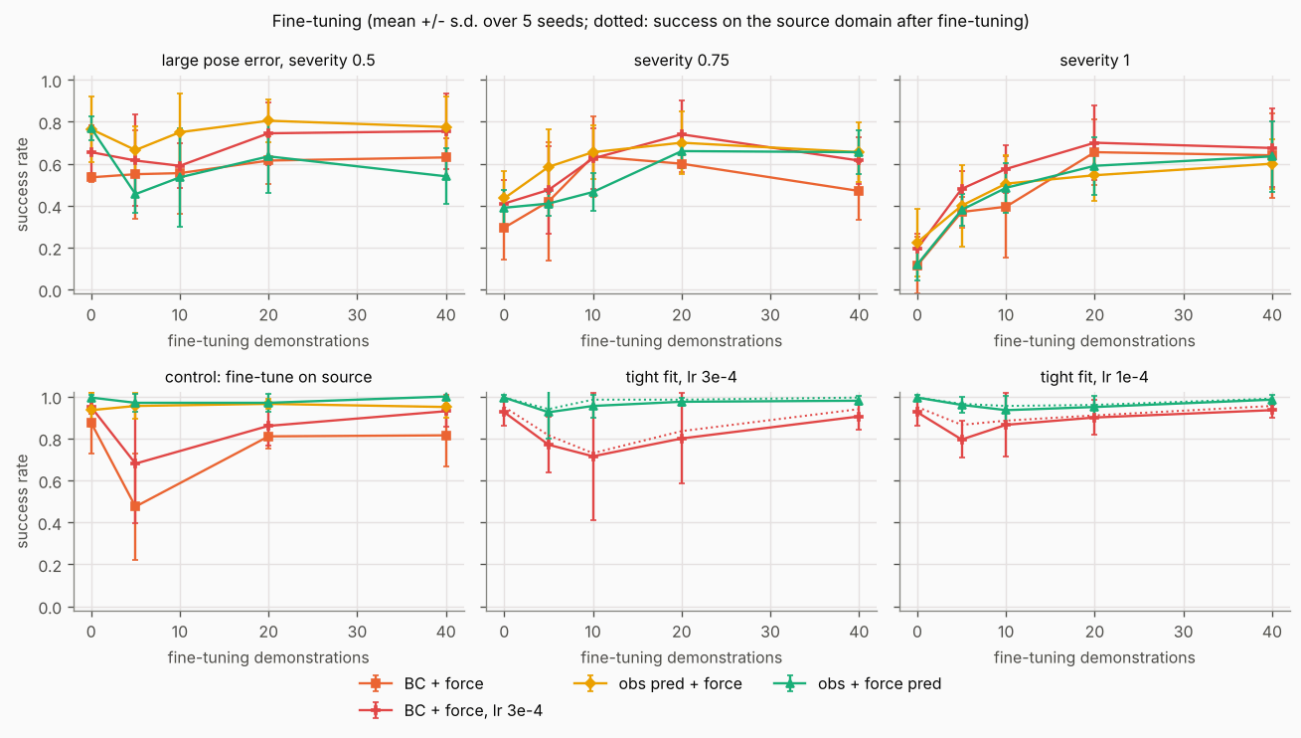


Figure 10. Fine-tuning from the E checkpoints (mean and s.d. over 5 seeds, 40 evaluation episodes per point). Top: the search shift at three severities. Bottom left: control, fine-tuning on source-domain demonstrations. Bottom middle and right: the v0.1 shift (tight fit) at two learning rates; dotted lines show success on the source domain after fine-tuning.

Adaptation. At severity 1, zero-shot success was low for every family (0.11 to 0.22), and 40 demonstrations raised it to 0.60 to 0.68 (gain +0.38 to +0.52); no family reached 0.9 within 40 demonstrations, and the families did not differ clearly (area under the gain curve 0.29 to 0.42, s.d. 0.06 to 0.16). At severity 0.5, where zero-shot success was already 0.54 to 0.77, fine-tuning on search demonstrations *lowered* the full model's success (0.77 to 0.45 after 5 demonstrations, 0.54 after 40).

The fine-tuning dip. In v0.1, fine-tuning on 5 to 20 demonstrations lowered success before 40 restored it. The source-domain control shows that this is a small-data fine-tuning effect of behaviour cloning, not a response to the shift: fine-tuning behaviour cloning on 5 *source* demonstrations lowered its *source* success from 0.88 to 0.48 (0.95 to 0.68 for the tuned variant), whereas the predictive models stayed at 0.95 to 0.97. On the v0.1 tight-fit shift, the tuned behaviour-cloning policy dipped from 0.92 to 0.72 at learning rate 3e-4 (minimum after 10 demonstrations) and to 0.80 at 1e-4, with a similar drop on the source domain (0.95 to 0.73 and 0.86; forgetting); the full predictive model, trained on the same data, stayed at 0.92 to 0.99 at both rates. The auxiliary losses, which keep being trained during fine-tuning, appear to protect the policy from over-fitting a handful of demonstrations.

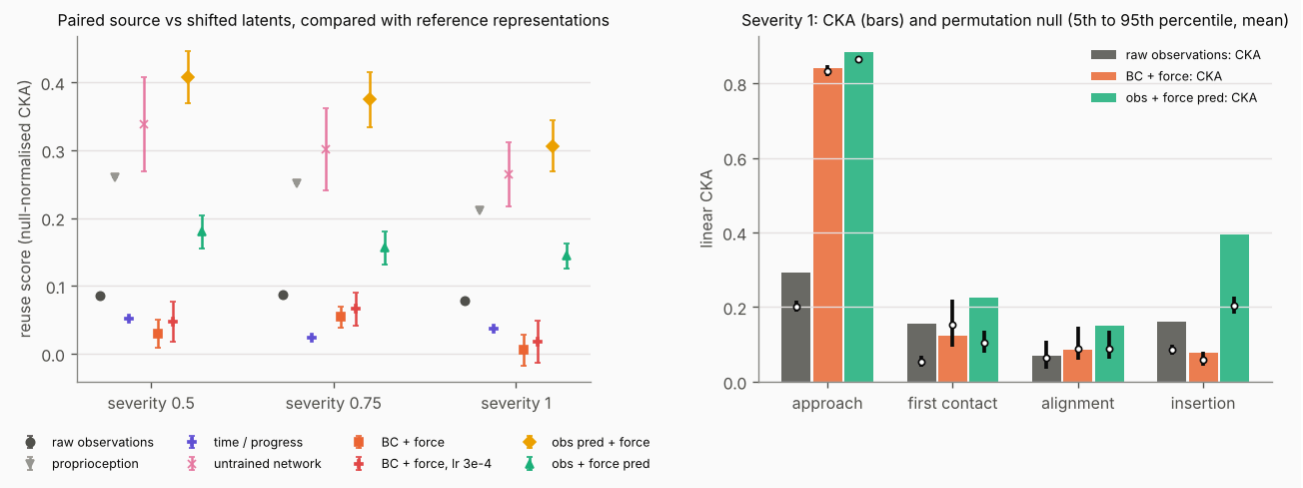


Figure 11. Left: reuse score (null-normalised, phase-matched CKA between paired source and shifted latents) for the trained models (mean and s.d. over 5 seeds) and for reference representations: standardised raw observations, proprioception, a time/progress clock, and the untrained network (same architecture and seeds). Right: severity 1, CKA per phase (bars) with the permutation null (black: 5th to 95th percentile; circle: mean).

Representation stability. Under the search shift, the paired source and shifted latents of the behaviour-cloning models were barely more similar than unpaired ones (reuse 0.00 to 0.07), the full model reached 0.14 to 0.18 and the observation-prediction model 0.31 to 0.41; the untrained network scored 0.26 to 0.34 and proprioception alone 0.21 to 0.26. Training therefore did not make the representations more stable than an untrained network of the same architecture, except for the model trained to predict observations without a force target. The score was robust to the analysis choices (2 or 8 resampling points, or excluding the approach phase, changed family averages by at most 0.07 and did not change the order of behaviour-cloning and predictive families). Under the v0.1 tight-fit shift, by contrast, the world-model latents were more similar than the raw observations in the contact phases (Appendix C). Representation stability is thus shift-dependent and, under a shift that changes the required behaviour, not a property that training produced.

Correlation with adaptation (H4). Across the 20 trained models, reuse (averaged over severities) and adaptation gain (area under the gain curve) were not positively associated: Spearman rho -0.34 [-0.74, +0.20]; per severity -0.30, -0.05 and -0.35, all intervals including 0. Reuse was associated with zero-shot success (0.55 [0.12, 0.84]). Within families n = 5 and the intervals span almost the whole range.

Decoding (H2). The contact mode was decoded from frozen latents with balanced accuracy 0.66 (full model) to 0.70 (behaviour cloning) within the source domain, so predictive training did not increase linear decodability. With held-out episode pairs, accuracy was 0.81 to 0.86 on the source members and 0.78 to 0.83 on the shifted members. The higher values relative to within-source cross-validation are explained by class support: the source analysis set contains only three jam steps (out of 4,945), whose near-zero recall counts as one sixth of the balanced accuracy, while the held-out-pair evaluation uses the classes present in both test sets (no jam class). The same explains v0.1's higher cross-domain than within-source accuracy.

Latent ablation. Clamping a quarter of the latent dimensions to their mean during control lowered success by 0.30 to 0.64 in every model, whichever dimensions were chosen; over 5 seeds, contact-ranked dimensions were not more harmful than random or movement-ranked ones (contact minus random: -0.01 to +0.13, all intervals including 0). The

coordinate-wise ablation did not isolate a contact-specific contribution. It does not distinguish distributed contact information from general sensitivity of the latent, and v0.1's statement that contact information "is distributed" is withdrawn.

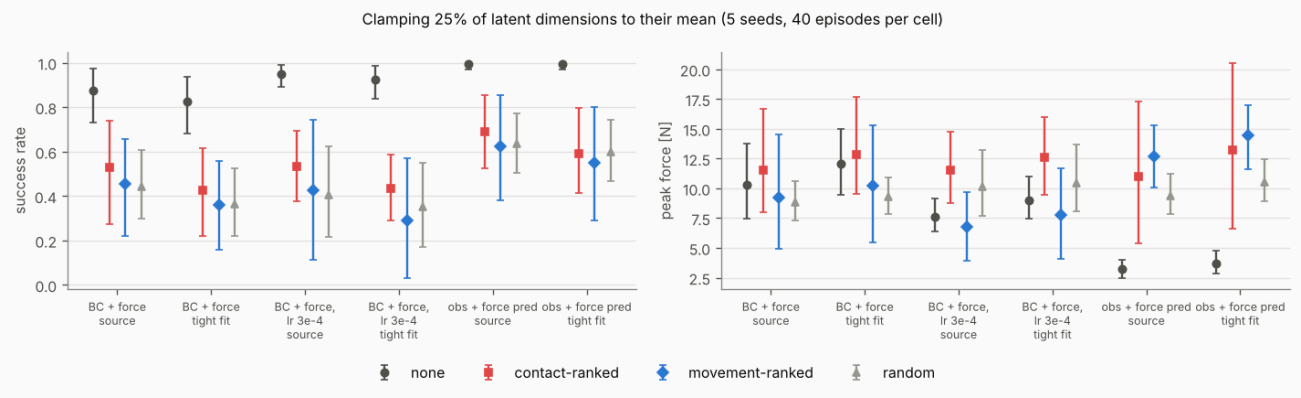


Figure 12. Latent ablation over 5 seeds (40 episodes per cell; "random" averages three draws per seed).

5.8 Outcome summary

	Outcome	Main evidence
H1	Not supported as stated	effect of force depended on its role; input effects small or inconclusive, harmful under sensor faults
H2	Not supported	within-source decoding 0.66 (full model) to 0.70 (behaviour cloning)
H3	Shift-dependent; not supported for the search shift	world-model latents above raw observations in contact phases for tight fit (v0.1 data); below the untrained network for search
H4	Not supported (exploratory)	rho -0.34 [-0.74, +0.20] across 20 trained models; similar gains for all families
H5	Supported (onset sensitivity)	onset window +1.8 N vs +0.4 N for a later window removing similar signal
Q1	Prediction helps; force mainly as a target	+0.18 vs BC with force; force target without input +0.22
Q2	Not failure data, not generic reconstruction; smaller vs a tuned baseline	Section 5.2
Q3	Mostly through commanded compliance	equal forces under fixed compliance
Q4	Pattern holds; value of force input depends on the task	G1: force target only 0.98; G2: force input needed, force target still +0.34

6. Discussion

Force as a training signal. The clearest result is that force was most useful as something to predict. A model that never receives force but is trained to predict it, together with the next observation, reached 0.96 success and 6.2 N, and was among the most robust models under sensor faults (0.98, tied with fault-aware training). Live force input then gave a modest further reduction of contact force when the sensor was healthy. This suggests a practical ordering for systems where force sensing is available during data collection: use force first as a training target, and add it as an input only together with fault-aware training or sensor monitoring.

Prediction as regularisation of the encoder. The controls point to a regularisation-like mechanism acting on the shared encoder: the gradient of the auxiliary losses dominates that of the action loss, the success gain is flat over a wide range of weights, a separate

predictive core works as well as the shared one, and generic reconstruction gives part of the effect. But the content of the target matters too: predicting the future was better than reconstructing the present, and force targets helped beyond observation targets. In a small-data regime (216 successful training demonstrations), a dense, physically meaningful prediction target seems to give the encoder features that make behaviour cloning more reliable across seeds. I cannot tell from these experiments whether the same holds for large datasets, where behaviour cloning is less data-limited.

What "lower forces" means. The force reduction came mostly from the compliance the policy chose. That is a legitimate way to lower forces (it is what the demonstrator does), but it means the result is about learning a contact strategy, not about a better internal model of contact as such. Under fixed compliance, the predictive model still succeeded more often, and its own adaptive compliance was better than any fixed value.

Onset sensitivity. The policies depended on the force reading mostly in the first 0.3 s of contact, the moment where the policy has to slow down and switch to compliant behaviour. This is a hypothesis for hardware rather than a specification: it suggests testing whether low-latency, reliable force sensing at contact onset matters more than continuous high-quality force sensing, with the same paired-intervention design on a real robot.

What did not survive. Two v0.1 claims did not hold up under matched data and architecture: that force as an input harms behaviour cloning, and that the onset window accounts for most of the total force effect. The representation-reuse analysis did not provide a strong test of whether reuse predicts adaptation (Section 5.7).

7. Limitations

- **Simulation only.** MuJoCo soft contacts make force magnitudes depend on solver settings; results compare conditions within the simulator, not absolute forces on hardware.
- **Idealised force.** The force channel is the summed contact wrench, not a wrist sensor model or force estimated from motor currents.
- **Small, simple tasks.** A planar, rigid, single-peg insertion and a hole-search variant. "Vision" is a low-dimensional pose estimate, not images. Results may differ with images, deformable parts or multi-contact tasks.
- **Scripted demonstrators.** Two demonstrators (privileged and non-privileged), both state machines with similar strategies; human demonstrations are more variable.
- **Small data and small models.** 300 demonstrations, 61k parameters, 1,500 updates. The relative value of auxiliary losses may shrink with more data.
- **Five seeds.** Intervals include training-run variation, but with 5 seeds they are themselves uncertain; several behaviour-cloning comparisons remain inconclusive for this reason.
- **Learning rates.** The default learning rate (1e-3) was inherited from v0.1. A learning rate of 3e-4 helped behaviour cloning; a full hyperparameter search for every condition was not run.
- **Post-hoc analyses.** All experiments in this revision were designed after reading the reviews and after seeing the v0.1 results. They were specified in configuration files

before running, but they are not pre-registered in the sense of the original hypotheses.

8. Future research

1. **Actuator-derived force.** Replace the ideal contact wrench with force estimated from simulated motor currents (friction, gearing, drift), then with real joint-torque data, and test whether force as a training target still beats force as an input.
2. **Data scale.** Vary the number of demonstrations over two orders of magnitude to see where the auxiliary-loss benefit disappears.
3. **Fault-aware sensing.** Combine force prediction with sensor-health estimation, and test unseen fault types.
4. **Hardware onset test.** Repeat the onset-window intervention on a real insertion setup.
5. **Images and richer models.** Image observations, stochastic latent dynamics and diffusion or flow action heads, keeping the controlled design.

9. Reproducibility

Code, configurations, per-episode results and figures are in the `contact-state-reuse` repository, release v0.2.0 (exact package versions in `requirements-lock.txt`; Python 3.13, MuJoCo 3.16, PyTorch 2.14, CPU only). Simulator settings are listed in Appendix A. All v0.2 experiments run with

```
pip install -r requirements-lock.txt && pip install -e . --no-deps
bash scripts/run_revision.sh          # about 6 hours on 2 CPU cores
python scripts/analyze_force_input.py --run outputs/E_factorial_medium/run1 --out
results/revision
python scripts/summarize_revision.py --out results/revision
python scripts/make_revision_figures.py --results results/revision --out
docs/whitepaper/figures
```

Every number in this paper comes from `results/revision/` (per-episode files under `runs/`, tables in `SUMMARY.md` and `*.csv`, key numbers in `key_numbers.json`); Appendix D maps figures and tables to files. Runs are deterministic on CPU for a given seed. v0.1 results are kept in `results/medium/`.

References

1. Agarwal, R., Schwarzer, M., Castro, P. S., Courville, A., and Bellemare, M. G. (2021). Deep reinforcement learning at the edge of the statistical precipice. *NeurIPS*.
2. Bernardi, S., Benna, M. K., Rigotti, M., Munuera, J., Fusi, S., and Salzman, C. D. (2020). The geometry of abstraction in the hippocampus and prefrontal cortex. *Cell* 183(4), 954-967.
3. Davari, M., Horoi, S., Natik, A., Lajoie, G., Wolf, G., and Belilovsky, E. (2023). Reliability of CKA as a similarity measure in deep learning. *ICLR*.
4. de Haan, P., Jayaraman, D., and Levine, S. (2019). Causal confusion in imitation learning. *NeurIPS*.

5. Fang, H., Tang, S., Mei, M., Qin, H., He, Z., Chen, J., Ying, F., Wang, C., Liu, W., He, Z., Lu, C., and Wang, S. (2026). Force Policy: Learning hybrid force-position control policy under interaction frame for contact-rich manipulation. *Robotics: Science and Systems (RSS)*.
6. Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., and Davidson, J. (2019). Learning latent dynamics for planning from pixels. *ICML*.
7. He, Z., Fang, H., Chen, J., Fang, H.-S., and Lu, C. (2025). FoAR: Force-aware reactive policy for contact-rich robotic manipulation. *IEEE Robotics and Automation Letters* 10(6), 5625-5632.
8. Jaderberg, M., Mnih, V., Czarnecki, W. M., Schaul, T., Leibo, J. Z., Silver, D., and Kavukcuoglu, K. (2017). Reinforcement learning with unsupervised auxiliary tasks. *ICLR*.
9. Kornblith, S., Norouzi, M., Lee, H., and Hinton, G. (2019). Similarity of neural network representations revisited. *ICML*.
10. Laskey, M., Lee, J., Fox, R., Dragan, A., and Goldberg, K. (2017). DART: Noise injection for robust imitation learning. *CoRL*.
11. Lee, M. A., Zhu, Y., Srinivasan, K., Shah, P., Savarese, S., Fei-Fei, L., Garg, A., and Bohg, J. (2019). Making sense of vision and touch: Self-supervised learning of multimodal representations for contact-rich tasks. *ICRA*.
12. Owen, A. B. (2007). The pigeonhole bootstrap. *Annals of Applied Statistics* 1(2), 386-411.
13. Todorov, E., Erez, T., and Tassa, Y. (2012). MuJoCo: A physics engine for model-based control. *IROS*.
14. Towers, M., Kwiatkowski, A., Terry, J., et al. (2024). Gymnasium: A standard interface for reinforcement learning environments. arXiv:2407.17032.
15. Wang, M., Yue, Z., Lu, R., Li, Y., Wang, Z., Pan, G., Dong, K., Cheng, J., Cheng, Y., and Liu, H. (2026). PhaForce: Phase-scheduled visual-force policy learning with slow planning and fast correction for contact-rich manipulation. arXiv:2603.08342 (preprint).
16. Wen, C., Lin, J., Darrell, T., Jayaraman, D., and Gao, Y. (2020). Fighting copycat agents in behavioral cloning from observation histories. *NeurIPS*.
17. Xie, W., and Correll, N. (2025). Towards forceful robotic foundation models: a literature survey. arXiv:2504.11827 (preprint).
18. Yu, J., Liu, H., Yu, Q., Ren, J., Hao, C., Ding, H., Huang, G., Huang, G., Song, Y., Cai, P., Zhang, W., and Lu, C. (2025). ForceVLA: Enhancing VLA models with a force-aware MoE for contact-rich manipulation. *NeurIPS*.
19. Zang, Y., Zheng, Y., Nie, X., Zheng, Y., Tian, S., Gu, S., Gao, C., Wang, Z., Yan, S., and Ding, W. (2026). TacForeSight: Force-guided tactile world model for contact-rich manipulation. arXiv:2606.11184 (preprint).

Appendix A. Settings

Setting	Value
Physics	MuJoCo 3.16, 500 Hz, implicit-fast integrator, elliptic friction cones, soft contacts (solref time constant 6 ms divided by the square root of the contact-stiffness factor, damping ratio 1; solimp 0.95, 0.99, 0.5 mm)
Control	20 Hz, 25 physics steps per action, episodes of at most 160 steps (8 s)
PD actuators	2,000 N/m (translation) and 6 N m/rad (rotation), scaled by 0.25 to 1.0 with the compliance command; damping ratio 0.8; set-point wind-up limited to 12 mm / 0.15 rad from the current pose
Goal / limits	25 mm depth; force violation above 10 N; damage termination above 40 N; jam after 1.5 s in contact with mean force above 2 N and under 0.5 mm progress
Training data	300 episodes per demonstrator, 20 % perturbed strategies, DART sigma 0.1; successful episodes only for every loss (216 training, 36 validation)
Network	fusion encoder (embedding 32 per modality), GRU 66, heads 66-66; 61,469 parameters in every E condition (reconstruction control 66,829; separate-core control 88,001; wide BC 116,097)
Optimiser	AdamW, learning rate 1e-3 (or 3e-4, 1e-4 where stated), cosine decay to 5 % of the initial rate, weight decay 1e-4, gradient clipping 1.0
Batch / updates	16 episodes / 1,500 (3,000 in the long-training control)
Prediction targets	next observation (exteroception and proprioception, 11 values, MSE); force readings 1 to 5 steps ahead (Gaussian NLL)
Fault-aware training	one extra copy of every training episode with force noise 1 to 2 N, bias 1 to 2 N (random direction), 2-step delay, 10 % dropout on the input; targets unchanged
Fine-tuning (R)	from the pre-trained checkpoint; 5, 10, 20, 40 nested episodes; 300 updates; learning rate 3e-4 (also 1e-4); final checkpoint
Evaluation	seed 123 (never used for data); E, G: 50 episodes per domain and seed; T: 60; L, R: 40
Statistics	crossed seed x episode bootstrap, 2,000 resamples, percentile 95 % intervals

Appendix B. Use of simulator labels

Contact-mode labels are computed from privileged simulator state. In v0.2 they are used (1) to analyse latents and (2) to schedule the timing interventions of experiment T. No v0.2 policy receives them as input or as a training target. The v0.1 "simulator-supervised" condition is summarised in Appendix C.

Appendix C. The first version (v0.1) and what changed

v0.1 ran four experiments with 3 seeds: modality ablation (A), objective ablation (B), force timing (C) and representation reuse with adaptation (D). Its models were matched in total parameters by searching the hidden width (BC trunk 78 units, world model 66), behaviour cloning used successful episodes while prediction losses used all episodes, and intervals were bootstraps over evaluation episodes pooling the 3 seeds. Its results and figures remain in `results/medium/` and `docs/whitepaper/whitepaper_v1.md`.

v0.1 claim	Status in v0.2
Force as an input made behaviour-cloning policies worse (-0.07 to -0.23)	not replicated on a fixed trunk with identical data (+0.14 [-0.10, +0.38], inconclusive)
Force as input and prediction target raised success to 0.95 to 0.98 and cut peak force to about 3 N	replicated in direction (0.96, 5.5 N pooled; 3.5 N in the source domain); the force-target part carries most of the effect
Next-observation prediction raised success by 0.26 to 0.29	replicated with smaller effects (+0.10 to +0.23) on identical data
The onset window "accounts for 85 to 94 % of the full effect"	withdrawn as a decomposition; the onset sensitivity itself is replicated with stronger controls
Latent states were strongly reused (0.74 to 0.89)	for the tight-fit shift the learned latents were more similar than the raw observations (descriptive); under the search shift they were not more similar than an untrained network's
Reuse did not predict adaptation (rho 0.36)	v0.1 test was uninformative (ceiling); redesigned in Section 5.7
Contact information is distributed in the latent	withdrawn; the ablation does not isolate a contact-specific contribution

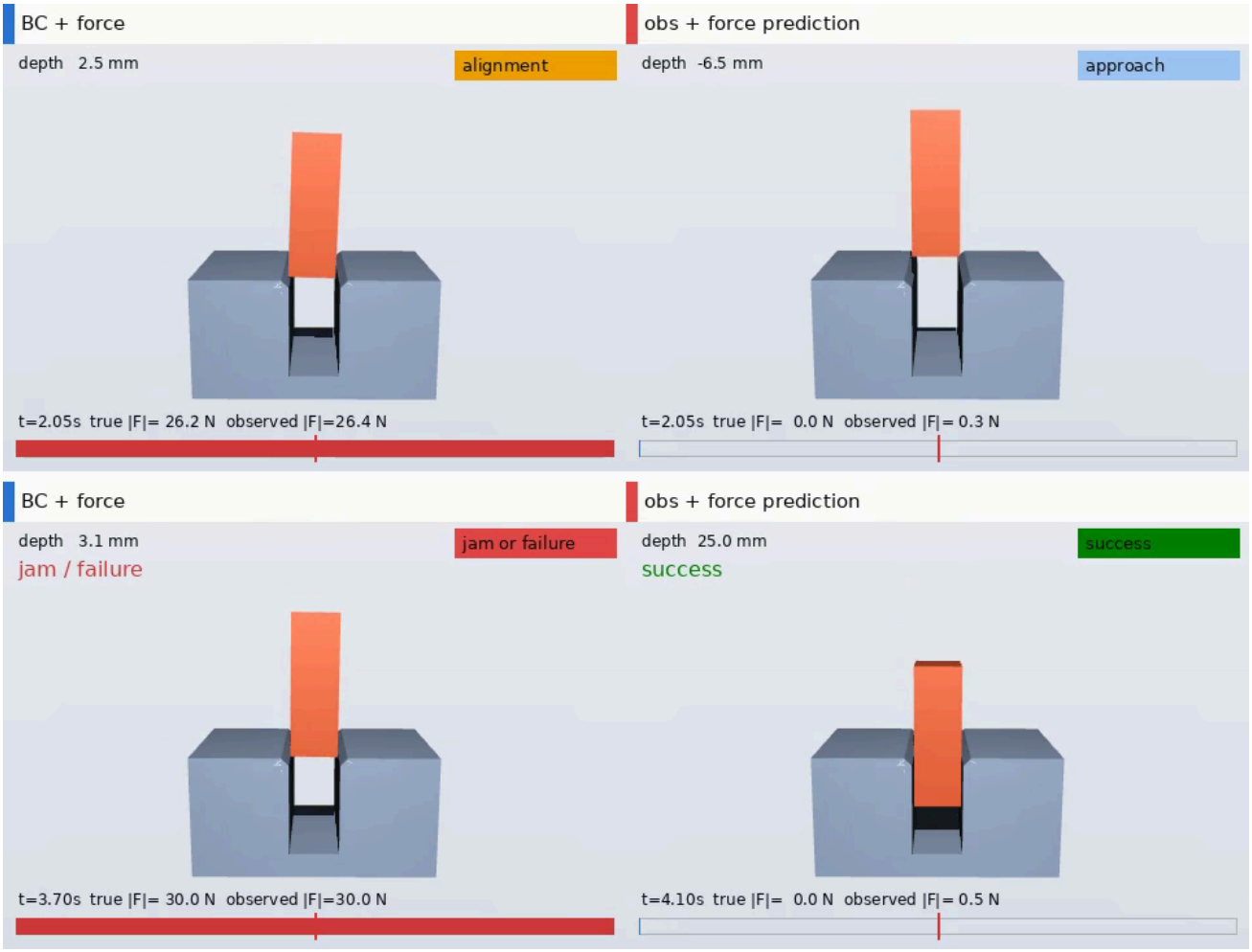
Why reuse was highest at intermediate severity in v0.1. The non-monotonic average came from the first-contact and insertion phases (for example sequence BC, insertion: 0.35, 0.80 and 0.50 at severities 0.33, 0.67 and 1.0; `results/revision/v1_reuse_by_severity_phase.csv`). It is not a support effect (the number of paired episodes is the same at every severity for approach, first contact and insertion, the phases that drive the pattern; alignment varies from 29 to 35) and not a pairing error (all non-clearance parameters of the paired episodes are identical). The same score computed on the raw standardised observations, with no model involved, shows the same pattern (0.67, 0.75 and 0.70; first contact 0.52, 0.71, 0.61; insertion 0.37, 0.46, 0.44), and so does proprioception alone (0.60, 0.73, 0.70; `results/revision/v1_reference_reuse.csv`). The non-monotonicity is therefore a property of the paired demonstration trajectories at these three severities, not of the learned representations. Compared with the raw observations, the two v0.1 world models' latents were more similar in the three contact phases at every severity (for example insertion 0.65 to 0.84 vs 0.37 to 0.46) but slightly less similar in approach; sequence BC was below the raw observations in first contact at every severity and above them in alignment. This comparison was missing in v0.1; v0.2 reports it for every analysis.

Appendix D. Where each number comes from

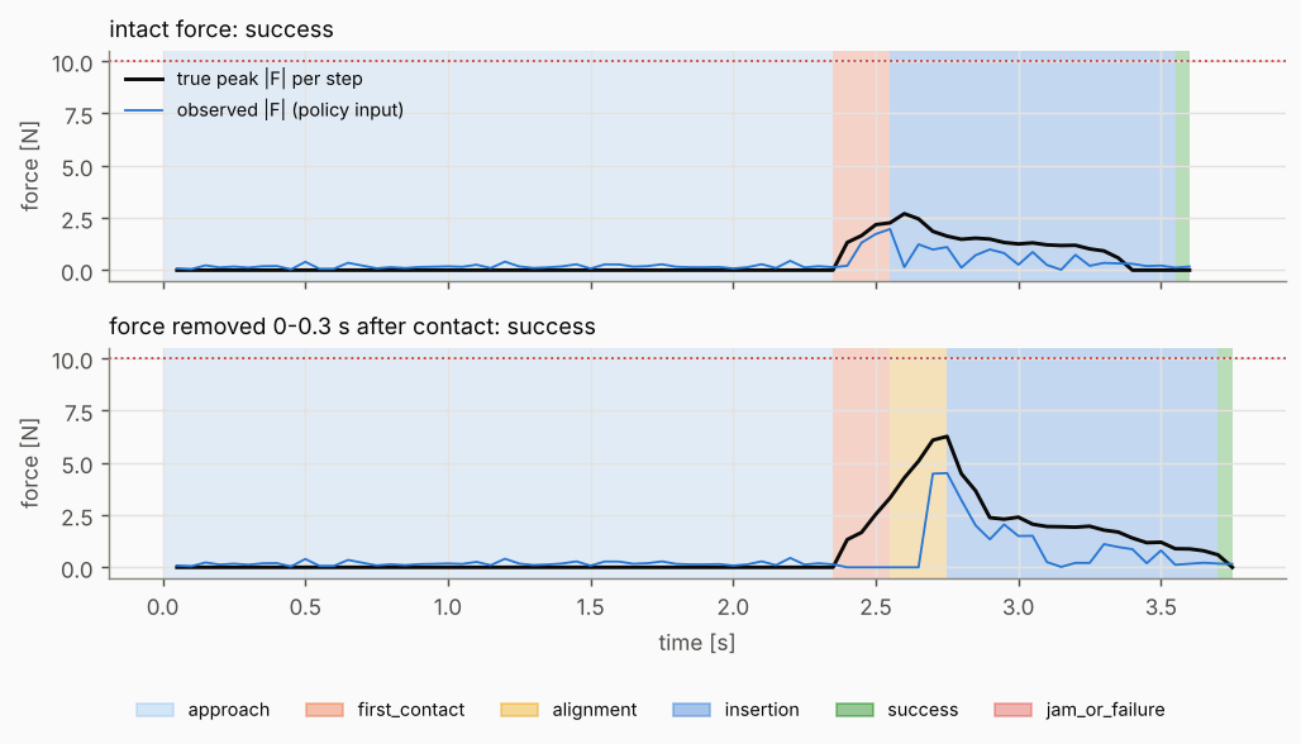
Item	File in `results/revision/`
Table 1, Figures 2 to 4	`E_conditions.csv`, `E_contrasts.csv` (per-episode data: `runs/E_factorial/episode_metrics.csv`)
Gradient norms, training losses	`E_training.csv`; full histories `runs/E_factorial/training_histories.json`
First-contact AUROC and baselines	`E_auroc.csv`
Figure 5	`force_input_analysis.json`, `force_input_sensitivity.csv`
Table 2, Figure 6	`T_results.csv`, `T_contrasts.csv`
Figure 7	`E_behaviour.csv`, `T_compliance.csv`, `T_compliance_contrasts.csv`
Figure 8	`E_conditions.csv`, `E_fault_extra_conditions.csv`, `E_fault_extra_contrasts.csv`
Section 5.6, Figure 9	`G_nonpriv_*.csv`, `G_search_*.csv`, `scripted_baselines.csv`
Section 5.7, Figures 10 to 12	`R_*.csv`, `R_correlations.json`, `L_results.csv`, `L_contrasts.csv`
Appendix C	`v1_reuse_by_severity_phase.csv`, `v1_reference_reuse.csv`; v0.1 tables in `results/medium/`

Appendix E. Videos

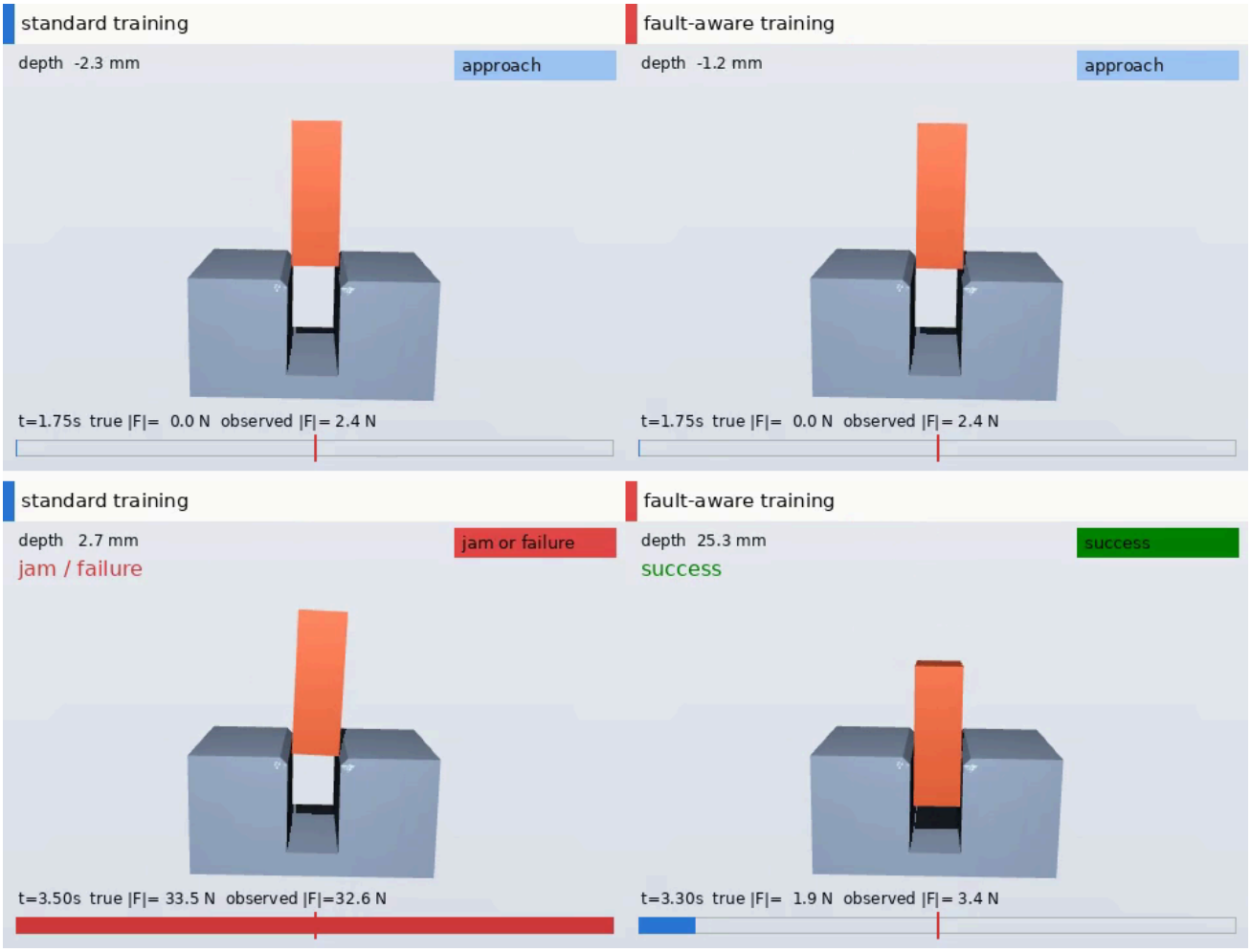
Each video shows two policies, or one policy with and without an intervention, on exactly the same evaluation episode (same geometry, physics and sensor noise). The bar at the bottom is the current contact force; the red tick marks 10 N. Files are in `assets/videos/` as MP4 and GIF, each with a force-trace figure (`*_force.png`). Videos made from v0.1 models are kept and marked "v0.1".



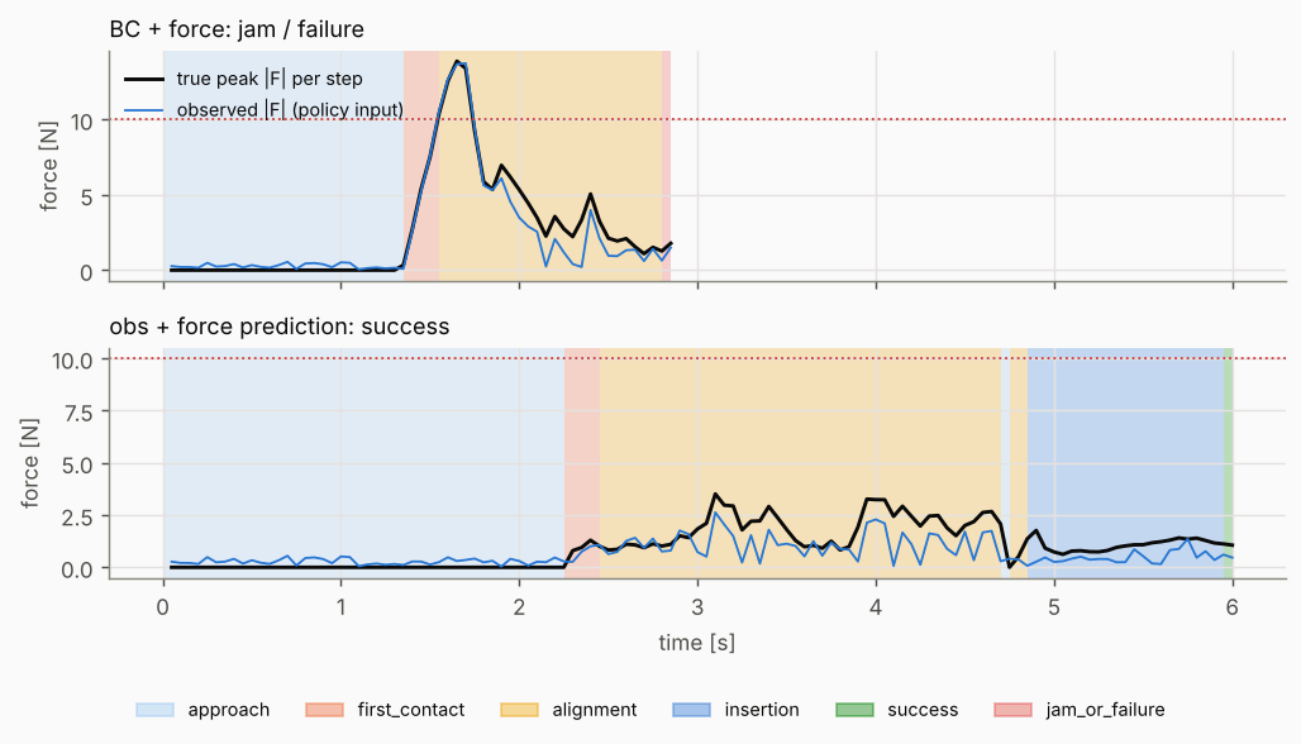
Video 1, `assets/videos/v2_bc_vs_predictive_tight_ep14.mp4`. Tight fit, same episode. Behaviour cloning with force (left) jams at 30 N; the model with observation and force targets (right) inserts at 1.3 N. Frames at mid-episode and at the end.



Video 2, `assets/videos/v2_onset_window_ep38.mp4`, force traces. The same predictive model with intact force (top, peak 2.7 N) and with the reading removed for the first 0.3 s of contact (bottom, 6.3 N). The observed force (blue) is 0 during the window while the true force (black) rises.



Video 3, `assets/videos/v2_fault_aware_ep27.mp4`. Sensor-fault domain. Standard training (left) jams at 33 N; fault-aware training (right) succeeds at 2.3 N.



Video 4, `assets/videos/v2_search_bc_vs_predictive_ep5.mp4`, force traces. Hole search: behaviour cloning with force (top) hits the top face at 14 N and is stopped as a jam after 1.5 s of pressing without progress; the predictive model (bottom) keeps the force below 4 N while sliding and inserts after 6 s.